

SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 7 OF 10

Microservices, Docker & Kubernetes

10 Questions | gRPC | Circuit Breaker | Distributed Tracing | K8s Probes | Canary Deployments

Microservices, Docker & Kubernetes

Q1: Monolith vs microservices — honest tradeoffs, not just benefits.

Every interview on microservices should begin with an honest comparison. Candidates who only recite microservice benefits signal they haven't actually operated them at scale.

Monolith advantages: Single deployment unit — no orchestration. In-process function calls — no network latency. Single DB transaction — no distributed consistency problem. Shared memory — no serialization overhead. Simpler debugging — one log stream, one stack trace. Faster development early on — no service boundary overhead.

Microservice advantages: Independent deployment — change and deploy User Service without touching Order Service. Independent scaling — scale only the bottleneck service. Technology heterogeneity — each service can use the right tool. Fault isolation — one service crashing doesn't take down the whole system. Team autonomy — each team owns their service end-to-end.

Microservice costs (rarely discussed honestly): Distributed systems complexity: network failures, partial failures, retries, timeouts. Operational overhead: each service needs its own CI/CD, monitoring, alerting, scaling config. Data consistency: no cross-service ACID transactions. Latency: what was a function call is now a network round trip (add 1-50ms per hop). Debugging: requires distributed tracing across services. Testing: integration testing is much harder.

The real decision: Start with a monolith. Extract services when you have a clear pain point: a specific component needs independent scaling, a team boundary emerges, or a component has a genuinely different deployment lifecycle. Don't start with microservices — you won't know your service boundaries until you've built the product.

```
# When a monolith becomes painful:
# 1. Deployment coupling: changing auth requires deploying everything
# 2. Scaling bottleneck: video processing needs 32 cores, API needs 2
# 3. Team coupling: 10 teams all modifying the same codebase
# 4. Tech requirement: ML service needs Python, core needs Go

# Strangler Fig pattern: extract services incrementally
# 1. New functionality goes into the new service
# 2. Route specific endpoints to new service via API gateway
# 3. Gradually migrate existing functionality
# 4. Monolith shrinks; services grow
# Never do a big-bang rewrite
```

Q2: Inter-service communication — sync vs async, REST vs gRPC vs messaging.

How services talk to each other is one of the most consequential architectural decisions. The choice affects latency, coupling, reliability, and observability.

Synchronous REST/HTTP: Simple, universal, easy to debug with curl. Best for: client-facing APIs, request-response patterns where the result is needed immediately, low-frequency inter-service calls. Downside: temporal coupling — both services must be available. Failure propagation — if Service B is slow, Service A slows or times out.

gRPC: HTTP/2-based RPC framework using Protocol Buffers. Strongly typed contracts (proto files), bi-directional streaming, efficient binary serialization (3-10x smaller than JSON), code generation for clients. Best for: high-frequency internal service-to-service calls, streaming, polyglot environments. Your Kafka pub/sub pipeline for Deepta AI sounds like a gRPC candidate internally.

Async messaging (Kafka/RabbitMQ): Services communicate by publishing events. Temporal decoupling — publisher doesn't wait for consumer. Absorbs load spikes (Kafka as buffer). Natural audit log. Best for: event-driven workflows, fan-out to multiple consumers, work queues, fire-and-forget operations.

```
# gRPC service definition (proto)
syntax = 'proto3';
service UserService {
  rpc GetUser (GetUserRequest) returns (UserResponse);
  rpc StreamUserEvents (UserRequest) returns (stream UserEvent);
}
message GetUserRequest { int64 user_id = 1; }
message UserResponse {
  int64 id = 1;
  string name = 2;
  string email = 3;
}

# Go server implementation
func (s *server) GetUser(ctx context.Context, req *pb.GetUserRequest)
(*pb.UserResponse, error) {
  user, err := s.db.GetUser(ctx, req.UserId)
  if err != nil { return nil, status.Errorf(codes.NotFound, 'user not found') }
  return &pb.UserResponse{Id: user.ID, Name: user.Name}, nil
}
```

Decision matrix: Need immediate result? → REST or gRPC. High frequency internal call with strict schema? → gRPC. Fire and forget, or multiple consumers? → Kafka. Job queue with retries? → RabbitMQ or Redis Streams. Real-time bidirectional? → WebSocket or gRPC streaming.

Q3: Circuit breaker pattern — how it works and why it's essential.

Without circuit breakers, a slow or failing downstream service causes your service to accumulate blocked threads/goroutines waiting for responses that never come — eventually exhausting your own resources and causing cascading failure across the entire system.

States: CLOSED (normal): requests flow through. Failure rate is monitored. OPEN (failing): requests are immediately rejected with an error — no attempt to call the downstream service. The circuit 'trips' when failure rate exceeds threshold. HALF-OPEN (testing): after a timeout, one request is allowed through. If it succeeds, circuit closes. If it fails, circuit stays open.

```
# Circuit breaker logic (conceptual Go)
type CircuitBreaker struct {
  state      State // CLOSED, OPEN, HALF_OPEN
  failures   int
  threshold  int
  timeout    time.Duration
  lastFailTime time.Time
}

func (cb *CircuitBreaker) Call(fn func() error) error {
  switch cb.state {
  case OPEN:
    if time.Since(cb.lastFailTime) > cb.timeout {
      cb.state = HALF_OPEN // allow one test request
    } else {
      return ErrCircuitOpen // fast-fail, no downstream call
    }
  }

  err := fn() // attempt the actual call
  if err != nil {
    cb.failures++
    cb.lastFailTime = time.Now()
    if cb.failures >= cb.threshold {
      cb.state = OPEN
    }
  }
}
```

```

    return err
}

// Success: reset
cb.failures = 0
cb.state = CLOSED
return nil
}

# Libraries: gobreaker (Go), resilience4j (Java), Polly (.NET)
# Service meshes (Istio, Linkerd) provide circuit breaking transparently

```

Fallback strategies: When the circuit is open, serve stale cached data, return a default response, degrade gracefully (show 'price unavailable' instead of failing the whole page), or queue the request for later processing.

Bulkhead pattern: Complement to circuit breaker. Isolate different downstream calls into separate goroutine pools/thread pools. If the payment service hangs, it can't consume all your goroutines and block calls to the user service. Named after ship compartments that prevent flooding from sinking the whole vessel.

Q4: Service discovery — how services find each other in a dynamic environment.

In a static environment, you hardcode hostnames. In Kubernetes or cloud environments, service instances are ephemeral — IPs change on restart, pods scale in and out. Service discovery solves dynamic routing.

Client-side discovery: Client queries a service registry (Consul, Eureka) to get available instances, then load-balances itself. Client has full control over load-balancing strategy. Downside: every client needs to implement discovery logic, in every language.

Server-side discovery: Client calls a load balancer or API gateway. The gateway queries the registry and routes to an instance. Client is simple — no discovery logic. This is what Kubernetes Services do natively.

Kubernetes DNS-based discovery: Every Kubernetes Service gets a DNS entry: service-name.namespace.svc.cluster.local. Pods call services by name — kube-dns resolves to the ClusterIP, kube-proxy routes to a healthy pod. No external registry needed for intra-cluster communication.

```

# Kubernetes service discovery – zero config needed
# Service A (in namespace 'default') calls Service B:
http://user-service/api/users/42
# or fully qualified:
http://user-service.default.svc.cluster.local/api/users/42

# Kubernetes automatically:
# 1. Routes to any healthy pod backing 'user-service'
# 2. Round-robin load balances across pods
# 3. Removes unhealthy pods from rotation (via readiness probes)
# 4. Updates routing as pods scale in/out

# Consul for multi-cluster or non-K8s environments
# Register service:
consul.agent.service.register({
  'name': 'user-service',
  'address': '10.0.1.5',
  'port': 8080,
  'check': {'http': 'http://10.0.1.5:8080/health', 'interval': '10s'}
})

```

Q5: Distributed tracing — how to debug a request across 10 services.

In a monolith, a slow request means looking at one log file and one stack trace. In microservices, a single user request might touch 10 services. Without distributed tracing, debugging is impossible at scale.

Trace and span: A trace is the full journey of a request across all services. Each service creates a span — a unit of work with a start time, duration, and metadata. Spans are connected by a trace ID propagated via HTTP headers. All spans for a trace can be visualized as a waterfall.

W3C Trace Context: Standard headers: traceparent (trace ID + span ID + flags) and tracestate (vendor-specific data). Always propagate these headers when making downstream calls. Any service that doesn't propagate them breaks the trace chain.

```
# OpenTelemetry – the standard SDK
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter

# FastAPI auto-instrumentation
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor
from opentelemetry.instrumentation.sqlalchemy import SQLAlchemyInstrumentor

FastAPIInstrumentor.instrument_app(app)      # auto-trace all routes
HTTPXClientInstrumentor().instrument()      # auto-trace outgoing HTTP
SQLAlchemyInstrumentor().instrument()       # auto-trace DB queries

# Custom span for business logic
tracer = trace.get_tracer('my-service')
with tracer.start_as_current_span('process-payment') as span:
    span.set_attribute('user.id', user_id)
    span.set_attribute('amount', amount)
    result = await charge_card()
    span.set_attribute('payment.status', result.status)
```

Backends: Jaeger (open source), Zipkin (open source), Tempo (Grafana), Datadog APM, AWS X-Ray. All accept OpenTelemetry format. Jaeger UI shows waterfall diagrams — you can see which service added latency and where errors occurred.

Correlation IDs: Even without full tracing, always generate a unique request ID at the entry point (API gateway or first service) and propagate it in all logs and downstream calls. Enables log correlation across services in a simple ELK stack setup.

Q6: Logging strategy in microservices — structured logging, log aggregation.

In a distributed system, logs from 20 service instances scattered across pods are useless without aggregation and structure. The goal: any engineer should be able to find a specific request's full journey in under 30 seconds.

Structured logging: Log JSON, not free-form strings. Every log line is a machine-parseable event with consistent fields. Enables filtering, aggregation, and alerting on specific fields without regex parsing.

```
# Bad – hard to query, parse, or alert on
logger.info(f'User {user_id} purchased {item} for ${amount}')
```

```
# Good – structured, queryable
logger.info('purchase.completed', extra={
    'user_id': user_id,
    'item_id': item.id,
    'amount': amount,
    'currency': 'USD',
    'trace_id': get_trace_id(),
    'request_id': request.state.request_id,
    'service': 'order-service',
    'environment': settings.env,
})
```

```
# Python structured logging with structlog
import structlog
log = structlog.get_logger()
log.info('purchase.completed', user_id=user_id, amount=amount)

# Go structured logging with zerolog or slog (stdlib 1.21+)
slog.Info('purchase completed',
    'user_id', userID,
    'amount', amount,
    'trace_id', traceID,
)
```

Log aggregation stack (ELK): Each service writes JSON logs to stdout. Fluentd/Filebeat collects from all pods. Elasticsearch indexes and stores. Kibana for search and dashboards. Alternatively: Loki (Grafana) — stores logs without indexing everything, much cheaper at scale.

Log levels discipline: DEBUG: developer detail, never in production. INFO: normal business events (request received, payment processed). WARN: unexpected but handled (retry succeeded, using fallback). ERROR: request failed, action needed. FATAL: process cannot continue. Never log passwords, tokens, PII, or credit card numbers — sanitize before logging.

Q7: What are the biggest challenges in microservices and how do you handle them?

This is a senior-level question. The expected answer goes beyond 'it's complex' — you should name specific failure modes and their mitigations with production experience.

Data consistency: No cross-service transactions. Mitigation: Saga pattern for multi-step operations, outbox pattern for reliable event publishing, eventual consistency with idempotent consumers, two-phase commit only for truly critical paths (avoid — it's slow and fragile).

Latency compounding: Each synchronous hop adds 5-50ms. A user request touching 5 services can have 200ms of added network latency. Mitigation: aggressive caching at each layer, async where possible, parallel calls (fan-out), consider merging chatty services.

Partial failure: Service B is deployed with a bug. It returns errors for 30% of requests. Your service must handle this. Mitigation: circuit breakers, retries with exponential backoff and jitter, timeouts on every outgoing call (never wait indefinitely), fallback responses.

Deployment complexity: 10 services, each with their own CI/CD pipeline, Dockerfile, K8s manifests, environment configs, secrets. Mitigation: Helm charts for templating, GitOps with ArgoCD for declarative deployment, standardized service templates.

Testing difficulty: Unit tests are easy. Integration tests across services are not. Mitigation: contract testing (Pact — each service defines the API contract and tests against it without running the real dependency), consumer-driven contract tests, dedicated integration test environments.

```
# Retry with exponential backoff and jitter (Go)
func retryWithBackoff(ctx context.Context, fn func() error) error {
    backoff := 100 * time.Millisecond
    for attempt := 0; attempt < 5; attempt++ {
        err := fn()
        if err == nil { return nil }

        // Check if context cancelled (don't retry if caller gave up)
        if ctx.Err() != nil { return ctx.Err() }

        // Jitter: avoid all retries hitting at the same time
        jitter := time.Duration(rand.Intn(100)) * time.Millisecond
        select {
        case <-time.After(backoff + jitter):
        case <-ctx.Done(): return ctx.Err()
        }
        backoff *= 2 // exponential: 100ms, 200ms, 400ms, 800ms, 1600ms
    }
}
```

```
    return fmt.Errorf('max retries exceeded')
}
```

Q8: Docker — architecture, layers, multi-stage builds, and optimization.

Understanding Docker's internals lets you write Dockerfiles that produce small, fast, secure images — not just images that work.

Layer caching: Each instruction in a Dockerfile creates a layer. Layers are cached by Docker. If a layer hasn't changed, Docker reuses the cached version. Order instructions from least-to-most frequently changing to maximize cache hits. COPY package.json before COPY . to avoid invalidating the dependency install layer on every code change.

Multi-stage builds: Use multiple FROM instructions in one Dockerfile. The first stage builds (compiles, installs deps). The final stage copies only the artifacts from the build stage — no build tools, no source code, no dev dependencies in the production image. Go binaries can go from 1GB build image to a 10MB scratch image.

```
# Multi-stage Go build
# Stage 1: build
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download                # cache this layer separately
COPY . .
RUN CGO_ENABLED=0 go build -o server ./cmd/server

# Stage 2: minimal runtime image
FROM scratch                        # empty base — ~0 MB
COPY --from=builder /app/server /server
COPY --from=builder /etc/ssl/certs /etc/ssl/certs # CA certs for HTTPS
EXPOSE 8080
ENTRYPOINT ["/server"]
# Final image size: ~8MB vs 300MB+ with go:alpine

# Multi-stage Python (FastAPI)
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt --target /app/deps

FROM python:3.12-slim
COPY --from=builder /app/deps /app/deps
COPY . /app
ENV PYTHONPATH=/app/deps
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Security best practices: Run as non-root user (USER 1001). Use specific image tags, never :latest in production. Scan images with Trivy or Snyk. Don't COPY .env or secrets into the image — use environment variables at runtime or K8s secrets.

Q9: Kubernetes — pods, deployments, services, probes, and rolling updates.

Kubernetes is the standard production platform for containerized microservices. You need to understand the core objects and how they compose to deploy a production service.

Pod: The smallest deployable unit — one or more containers sharing a network namespace and volumes. Pods are ephemeral — they're created and destroyed, never updated in place. You never manage pods directly — you manage Deployments.

Deployment: Declares the desired state: which image, how many replicas, resource limits. Kubernetes constantly reconciles actual state to match desired state. Rolling updates are built-in — new pods are created before old ones are removed.

Service: Stable network endpoint (ClusterIP, DNS name) in front of a set of pods. Selected by label selector. Abstracts away pod IP churn. Types: ClusterIP (internal only), NodePort (expose on node IP), LoadBalancer (cloud load balancer), ExternalName (DNS alias).

Liveness probe: Is the container alive? If it fails, K8s restarts the container. Check: process isn't deadlocked, event loop is responding. Use an /health/live endpoint that returns 200 if the process is running. Don't check downstream dependencies here — a slow DB shouldn't restart your healthy container.

Readiness probe: Is the container ready to receive traffic? If it fails, K8s removes the pod from the Service's load balancer (stops sending it requests). Check: DB connection pool initialized, caches warmed, dependencies reachable. A pod can be alive but not ready.

```
# Production deployment manifest
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # create 1 new pod before removing old
      maxUnavailable: 0 # never remove old pod before new is ready
  selector:
    matchLabels: { app: user-service }
  template:
    spec:
      containers:
        - name: user-service
          image: myregistry/user-service:v1.2.3
          resources:
            requests: { cpu: '100m', memory: '128Mi' }
            limits:   { cpu: '500m', memory: '512Mi' }
          livenessProbe:
            httpGet: { path: /health/live, port: 8080 }
            initialDelaySeconds: 10
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:
            httpGet: { path: /health/ready, port: 8080 }
            initialDelaySeconds: 5
            periodSeconds: 5
            failureThreshold: 2
      env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef: { name: user-service-secrets, key: db-url }
```

Q10: Blue-green and canary deployments — how they work and when to use each.

Zero-downtime deployments require traffic management strategies. Rolling updates are K8s default, but blue-green and canary offer more control for high-risk changes.

Rolling update: Default K8s strategy. Gradually replaces old pods with new ones. Simple. Risk: during the rollout, old and new versions serve traffic simultaneously — API must be backward compatible.

Blue-green deployment: Two identical environments: Blue (current live) and Green (new version). Deploy to Green, run smoke tests. Switch traffic entirely from Blue to Green (update load balancer or K8s service selector).

Instant rollback: switch back to Blue. Cost: doubles infrastructure during deployment. Best for: major version changes where mixed-version serving is unacceptable.

Canary deployment: Route a small percentage of traffic (1%, 5%, 10%) to the new version. Monitor error rates, latency, business metrics. Gradually increase percentage. Automatic rollback if metrics degrade. Best for: risky changes where you want real traffic validation before full rollout.

```
# Canary with Kubernetes – label-based traffic splitting
# Stable deployment: 9 replicas of v1
# Canary deployment: 1 replica of v2
# Service selects both via shared label 'app: user-service'
# 10% of traffic hits v2 (1 pod out of 10 total)

# With Argo Rollouts or Istio – precise percentage control
apiVersion: argoproj.io/v1alpha1
kind: Rollout
spec:
  strategy:
    canary:
      steps:
        - setWeight: 5      # 5% to new version
        - pause: {duration: 10m} # wait 10 min, watch metrics
        - setWeight: 25
        - pause: {duration: 10m}
        - setWeight: 50
        - pause: {duration: 5m}
        - setWeight: 100    # full rollout
      analysis:
        templates:
          - templateName: error-rate # auto-rollback if error rate > threshold
```

Feature flags: Decouple deployment from feature release. Deploy new code to 100% of servers but hide it behind a flag (LaunchDarkly, Unleash, custom Redis-backed flags). Enable for internal users first, then 1%, then all. Instant rollback without redeployment — just flip the flag.
